

Linked Lists — Deletion and Doubly Linked Lists

Lecture 09

Singly Linked List Deletion **Introduction to Doubly Linked List • Traversal • Insertion**

Data Structures and Algorithm Analysis

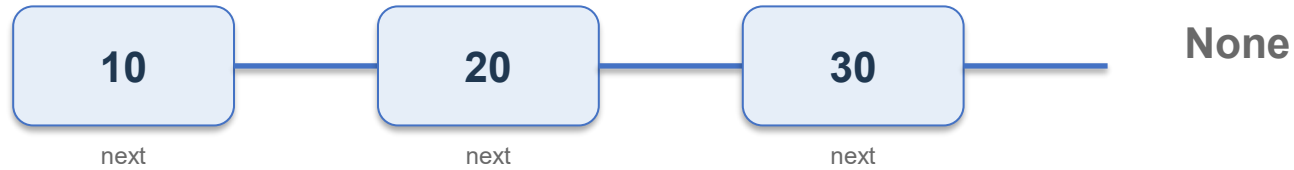
Learning Objectives

- Explain deletion in a singly linked list: beginning, end, position, and by value.
- Trace the link changes involved when a node is removed.
- Define a doubly linked list and explain the purpose of PREV and NEXT.
- Create and connect doubly linked-list nodes in Python and Java.
- Traverse a doubly linked list in forward and backward directions.
- Insert nodes at the beginning, end, and a specified position.
- Compare singly and doubly linked lists using time and space complexity.

Recap: Singly Linked List

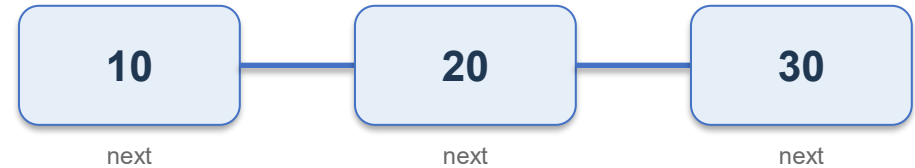
Each node has DATA + NEXT

HEAD gives the entry point; traversal moves only forward.



Singly Linked List — Deletion at Beginning

- Check whether the list is empty.
- Store the value of the current HEAD if the removed value is required.
- Move HEAD to HEAD.next.
- The old first node is no longer reachable from the list.



head = head.next

Before: 10 → 20 → 30

After: 20 → 30

Time: $O(1)$ | Auxiliary space: $O(1)$

Singly Linked List — Deletion at End

- Start at HEAD and find the second-last node.
- Stop when `current.next.next` is `None`.
- Save the last value if required.
- Set `current.next = None` to make the second-last node the new last node.

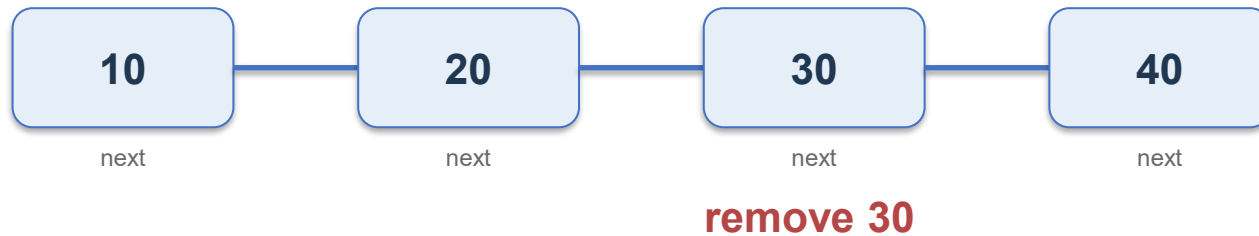


current = 20
current.next = None

Time: $O(n)$ without a tail/previous reference | Auxiliary space: $O(1)$

Singly Linked List — Deletion at a Position

Example: Delete the node at position 2 (0-based).



Reach the predecessor (20), then bypass the target:
 $20.\text{next} = 30.\text{next} \rightarrow 20.\text{next} = 40$

Result: $10 \rightarrow 20 \rightarrow 40 \rightarrow \text{None}$ | Time: $O(n)$ worst case

Singly Linked List — Deletion by Value

- Search for the target value while keeping a reference to the previous node.
- If the target is HEAD, move HEAD to HEAD.next.
- Otherwise, bypass the target with `previous.next = current.next`.
- If the target is absent, leave the list unchanged.

Python Example — Delete a Node by Value

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(10)
head.next = Node(20)
head.next.next = Node(30)
head.next.next.next = Node(40)

target = 30
current = head

while current.next is not None:
    if current.next.data == target:
        current.next = current.next.next
        break
    current = current.next

print("10 -> 20 -> 40 -> None")
```

Java Example — Delete a Node by Value

```
class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

void deleteByValue(int target) {
    if (head == null) return;

    if (head.data == target) {
        head = head.next;
        return;
    }

    Node current = head;
    while (current.next != null) {
        if (current.next.data == target) {
            current.next = current.next.next;
            return;
        }
        current = current.next;
    }
}
```

Deletion — Step-by-Step Dry Run

Delete 30 from: 10 → 20 → 30 → 40 → None

Step 1 current = 10

Step 2 current.next = 20 ≠ 30

Step 3 current = 20; current.next = 30

Step 4 20.next = 30.next (40)

Final: 10 → 20 → 40 → None

Introduction to Doubly Linked List

- A doubly linked list is a linked structure in which each node contains DATA, PREV, and NEXT.
- PREV points to the previous node; NEXT points to the next node.
- The first node has PREV = None and the last node has NEXT = None.
- Because links exist in both directions, traversal can proceed forward or backward.
- The extra PREV reference increases memory usage compared with a singly linked list.

Doubly Linked List — Node Representation

Each node stores two links.



None ← PREV

NEXT → None

HEAD → [10] ⇔ [20] ⇔ [30] ← TAIL

Python Representation of a Doubly Linked Node

```
class Node:
def __init__(self, data):
    self.data = data
    self.prev = None
    self.next = None

node1 = Node(10)
node2 = Node(20)
node3 = Node(30)

node1.next = node2
node2.prev = node1
node2.next = node3
node3.prev = node2

head = node1
tail = node3
```

Result: None ← 10 ⇄ 20 ⇄ 30 → None

Traversal of a Doubly Linked List — Forward

- Start at HEAD.
- Process current.data.
- Move using `current = current.next`.
- Stop when current becomes None.
- Forward traversal is conceptually the same as singly linked-list traversal.



current = current.next
10 → 20 → 30

Traversal of a Doubly Linked List — Backward

- Start at TAIL instead of HEAD.
- Process current.data.
- Move using `current = current.prev`.
- Stop when current becomes None.
- Backward traversal is a key advantage of the doubly linked structure.



current = current.prev
30 → 20 → 10

Python Example — Forward and Backward Traversal

```
# head and tail are already connected
current = head
print("Forward:")
while current is not None:
    print(current.data, end=" -> ")
    current = current.next
    print("None")

current = tail
print("Backward:")
while current is not None:
    print(current.data, end=" -> ")
    current = current.prev
    print("None")
```

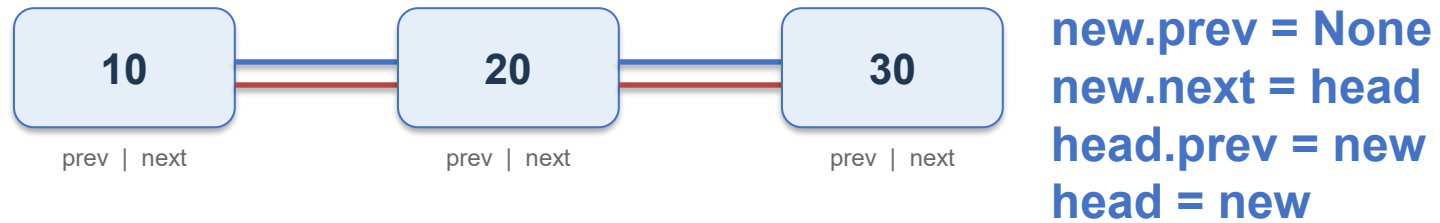
Example output: Forward: 10 -> 20 -> 30 -> None | Backward: 30 -> 20 -> 10 -> None

Insertion in a Doubly Linked List

- Insertion requires updating both PREV and NEXT references.
- At the beginning, connect the new node to the old HEAD and update the old HEAD.prev.
- At the end, connect the new node after TAIL and update TAIL.
- At a middle position, update four relevant links around the insertion point.
- Careful ordering of assignments prevents losing an existing connection.

Doubly Linked List — Insert at Beginning

Insert 5 before the current HEAD (10).



Result: None ← 5 ⇌ 10 ⇌ 20 ⇌ 30 → None

Time: O(1) when HEAD is available

Doubly Linked List — Insert at End

- Create the new node.
- Set `new.prev = tail`.
- Set `tail.next = new`.
- Move tail to the new node.
- If the list is empty, both head and tail become the new node.



`new.prev = tail`
`tail.next = new`
`tail = new`

With TAIL: $O(1)$

Doubly Linked List — Insert at a Position

Example: Insert 25 between 20 and 30.



new.prev = current
new.next = current.next
current.next.prev = new
current.next = new

Result: 10 ⇄ 20 ⇄ 25 ⇄ 30

Python Example — Insert at a Position

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

def insert_after(current, value):
    new_node = Node(value)
    new_node.prev = current
    new_node.next = current.next

    if current.next is not None:
        current.next.prev = new_node

    current.next = new_node
    return new_node

# 10 <-> 20 <-> 30
head = Node(10)
second = Node(20)
third = Node(30)
head.next = second; second.prev = head
second.next = third; third.prev = second

insert_after(second, 25)
# 10 <-> 20 <-> 25 <-> 30
```

Java Example — Insert at a Position

```
void insertAfter(Node current, int data) {  
    Node newNode = new Node(data);  
  
    newNode.prev = current;  
    newNode.next = current.next;  
  
    if (current.next != null) {  
        current.next.prev = newNode;  
    }  
  
    current.next = newNode;  
}
```

**For a known node: insertion itself is $O(1)$.
If the position must first be located, total cost is $O(n)$.**

Doubly Linked Insertion — Dry Run

Insert 25 between 20 and 30.

1. Create new node: 25

2. `new.prev = 20`

3. `new.next = 30`

4. `30.prev = 25`

5. `20.next = 25`

Final: 10 ⇌ 20 ⇌ 25 ⇌ 30

Singly vs Doubly Linked List

Feature	Singly Linked List	Doubly Linked List	Observation
Node links	NEXT only	PREV + NEXT	Doubly stores one extra reference
Forward traversal	Yes	Yes	Both support forward traversal
Backward traversal	No	Yes	Doubly can start from TAIL
Insert at beginning	O(1)	O(1)	Update HEAD; doubly also updates PREV
Insert at end	O(n) without tail	O(1) with TAIL	TAIL makes end insertion direct

Time and Space Complexity

- Singly deletion at beginning: $O(1)$; deletion at end/by value/position: $O(n)$ in the typical implementation.
- Doubly traversal: $O(n)$ in either direction because each node may be visited once.
- Doubly insertion at beginning: $O(1)$ with HEAD.
- Doubly insertion at end: $O(1)$ when TAIL is maintained.
- Doubly insertion at a known node: $O(1)$; locating that node by position/value may make the overall operation $O(n)$.
- Auxiliary space for these operations is $O(1)$, excluding the storage of the nodes themselves.
- Each doubly linked node uses more memory than a singly linked node because of PREV.

Applications of Doubly Linked Lists

- Browser history: move backward and forward between visited pages.
- Undo/redo systems: navigate to previous and next states.
- Music or image playlists: move forward and backward through items.
- LRU cache implementations: efficiently remove and move nodes when combined with a hash map.
- Deque-like structures: support operations at both ends when HEAD and TAIL are maintained.

Common Programming Mistakes

- Forgetting to update PREV when changing NEXT in a doubly linked list.
- Updating one link before saving the old connection and accidentally losing part of the list.
- Not handling an empty list.
- Not handling insertion/deletion at HEAD separately.
- Using a stale HEAD or TAIL after insertion/deletion.
- Assuming a linked list supports direct indexing like a Python list.
- Reporting measured execution time as Big-O; timing and asymptotic analysis are different.

Practice / Classroom Coding Tasks

- Write a Python program to delete the first node of a singly linked list and print the removed value.
- Write a Python program to delete the last node and display the final list.
- Delete the first occurrence of a user-entered value from a singly linked list.
- Create a doubly linked list of five values and traverse it forward and backward.
- Insert a new node at the beginning and end of a doubly linked list.
- Insert a node after a specified node in a doubly linked list and draw the links before and after.
- For each program, state time complexity and auxiliary space complexity.

Key Takeaways

- Singly linked-list deletion is performed by moving HEAD or bypassing a target node through NEXT references.
- A doubly linked list adds PREV, enabling bidirectional traversal.
- Doubly insertion requires coordinated updates to PREV and NEXT links.
- Maintaining TAIL makes insertion at the end $O(1)$.
- Knowing the target node versus locating it first is crucial when analyzing complexity.
- Correct pointer/link ordering is the central implementation skill.